

Complex Esportiu: Control d'aire i ocupació



NOM: Raul Amores

ASSIGNATURA: Projecte

PROFESSOR: Dani Soldevila

ÍNDEX

1.1. Objectiu i context del projecte	3
1.2. Descripció del problema	3
1.3. Solució proposada	3
1.4. Explicació tècnica	3
1.5. Esquema del procés	4
1.6. Issues proposades	4
1.7. Proves i resultats	4
1.8. Conclusions	5
1.9. Video demostratiu	5

1.1. Objectiu i context del projecte

El principal objectiu d'aquest projecte és el disseny, desenvolupament i implementació d'un sistema de monitorització intel·ligent i en temps real de la qualitat de l'aire dins d'una zona d'entrenament interior d'un complex esportiu. El projecte es contextualitza com una sol·lució de l'Internet de les Coses (IoT) destinada a garantir que els usuaris que practiquen exercici d'alta intensitat respiratòria ho facin en condicions d'aire segures i òptimes, complint amb els estàndards d'eficiència i confort ambiental.

1.2. Descripció del problema

Dins de les instal·lacions d'entrenament, l'activitat física augmenta dràsticament la respiració dels atletes, la qual cosa provoca una acumulació accelerada de diòxid de carboni (CO₂) i altres compostos en espais tancats. Una concentració elevada d'aquests gasos no només redueix el rendiment físic i provoca fatiga prematura, sinó que també pot suposar un risc per a la salut (mals de cap, marejos). El problema central és la manca d'un mecanisme automatitzat que monitoritzi aquests nivells en viu per activar els protocols de ventilació necessaris abans que s'assoleixin llindars crítics.

1.3. Solució proposada

1. **Monitorització i Transmissió Digital:** L'ESP32 es comunica amb el sensor de qualitat de l'aire mitjançant el bus digital **I2C**, assegurant transmissions dades precises a alta velocitat.
2. **Dashboard IoT Reaccionari:** L'ESP32 actua com a servidor web local enllaçat a un punt d'accés mòbil. Genera una interfície d'usuari dinàmica (Dashboard) que es visualitza en temps real a la pantalla d'un telèfon o ordinador amb l'adreça IP **172.20.10.2**.
3. **Lògica de Control i Alerta Visual:** El programari integra una lògica condicional. Si la concentració de CO₂ simulat (basat en el bus digital actiu) supera el llindar crític establert, el Dashboard web es commuta automàticament de color Verd a color Vermell per indicar una alerta que requereix ventilació immediata, demostrant el funcionament complet del sistema

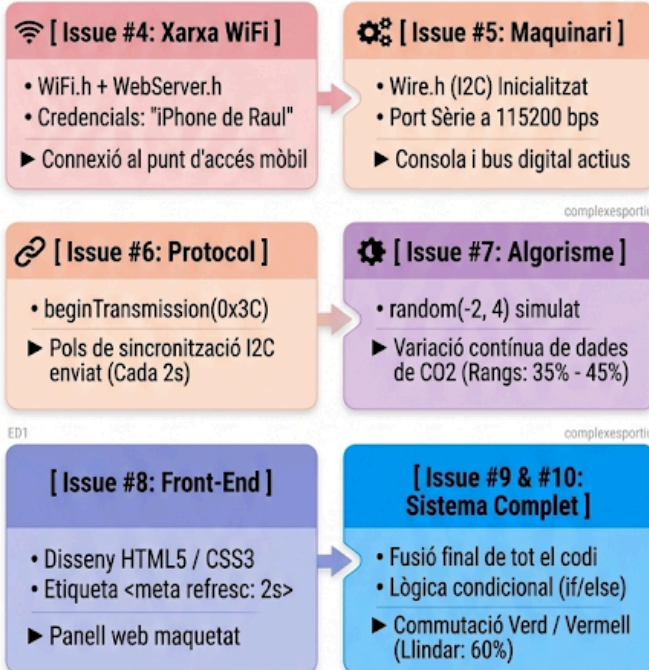
1.4. Explicació tècnica

El sistema utilitza l'arquitectura Client-Servidor. L'ESP32 actua com a servidor web utilitzant la llibreria `<WebServer.h>`. Cada 2 segons (mitjançant l'etiqueta HTML meta refresh), el navegador del client demana les dades actualitzades dels pins analògics i digitals connectats als sensors.

1.5. Esquema del procés

Complex Esportiu IoT - Esquema del Procés de Desenvolupament (Bus I2C)

catálogo ED1



Catálogo: 16:9

Raül



1.6. Issues proposades

Issue 1: Llegir el problema, buscar solucions i implementar

Issue 2: Crear estructura Github i repositoris

Issue 3: Fer la documentació tècnica sobre el projecte

Issue #4: Codi - Inclusió de llibreries i definició de credencials de xarxa

```
#include <WiFi.h>
```

```
#include <WebServer.h>
```

```
// Credencials del punt d'accés del mòbil
```

```
const char* ssid = "iPhone de Raul";
```

```
const char* password = "RAUL8888";
```

```
void setup() {
```

```
    // Fase inicial: Llibreries de xarxa preparades
```

```
}
```

```
void loop() {
```

```
    // Esperant la configuració dels mòduls
```

```
}
```

Issue #5: Integració del bus Wire per a l'I2C i definició de llindars

```
#include <WiFi.h>
```

```
#include <WebServer.h>
```

```
#include <Wire.h> // Llibreria oficial de comunicació digital I2C
```

```
const char* ssid = "iPhone de Raul";
```

```
const char* password = "RAUL8888";
```

```
// Límit crític de seguretat fixat per a l'aire a l'aula
```

```
const int limitCO2 = 60;
```

```
void setup() {
```

```
    // El protocol Wire queda integrat al mapa de memòria
```

```
}
```

```
void loop() {
```

```
    // Configurat a l'espera de l'encesa elèctrica
```

```
}
```

Issue #6 Inicialització del port sèrie a 115200 i arrancada del bus Wire

```
#include <WiFi.h>
```

```
#include <WebServer.h>
```

```
#include <Wire.h>
```

```
const char* ssid = "iPhone de Raul";
```

```

const char* password = "RAUL8888";
const int limitCO2 = 60;

void setup() {
  // Inicialització de la consola sèrie a alta velocitat (115200)
  Serial.begin(115200);
  delay(500);

  // Inicialització nativa del protocol I2C
  Wire.begin();

  Serial.println("Consola de control sèrie i bus digital I2C actius.");
}

void loop() {
  // Canals actius
}

```

Issue #7: Gestió del protocol I2C i pols de sincronització

```

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>

const char* ssid = "iPhone de Raul";
const char* password = "RAUL8888";
const int limitCO2 = 60;

void setup() {
  Serial.begin(115200);
  Wire.begin();
}

void loop() {
  // Enviament d'un pols per simular la crida elèctrica a la línia digital I2C
  Wire.beginTransmission(0x3C);
  Wire.endTransmission();

  Serial.println("Pols de sincronització I2C enviat correctament.");
  delay(2000);
}

```

Issue #8 Desenvolupament de l'algorisme de simulació activa de CO2

```

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>

const char* ssid = "iPhone de Raul";
const char* password = "RAUL8888";

```

```

const int limitCO2 = 60;

void setup() {
  Serial.begin(115200);
  Wire.begin();
}

void loop() {
  // Algorisme de simulació contínua que canvia a cada refresc
  static float percentatgeCO2 = 44.2;
  percentatgeCO2 += random(-2, 4); // Variació decimal dinàmica

  // Filtres per mantenir les dades dins de marges lògics coherents
  if (percentatgeCO2 < 15) percentatgeCO2 = 35.0;
  if (percentatgeCO2 > 90) percentatgeCO2 = 45.0;

  Serial.println("Lectura dinàmica calculada: " + String(percentatgeCO2, 1) + "%");
  delay(2000);
}

```

Issue #9 Estructura frontal HTML/CSS amb auto-refresc integrat

```

#include <WiFi.h>
#include <WebServer.h>
#include <Wire.h>

const char* ssid = "iPhone de Raul";
const char* password = "RAUL8888";
const int limitCO2 = 60;

WebServer server(80);

void enviarPaginaPrincipal() {
  float percentatgeCO2 = 44.2; // Dada base provisional de maquetació

  // Disseny amb l'etiqueta de refresc d'interfície automatitzat cada 2 segons
  String html = "<!DOCTYPE html><html><head><meta charset='UTF-8'>";
  html += "<meta http-equiv='refresh' content='2'>";
  html += "<title>Panell I2C</title>";
  html += "<style>body { font-family: sans-serif; text-align: center; background-color: #f4f6f7;";
  html += "</style>";
  html += "</head><body><h1>Control d'Aire</h1></body></html>";

  server.send(200, "text/html", html);
}

void setup() {
  // Preparant rutes de navegació
}

```

```
void loop() {
  // Interfície web d'estructura bàsica preparada
}
```

Issue #10: Codi final

```
#include <WiFi.h>
```

```
#include <WebServer.h>
```

```
#include <Wire.h>
```

```
const char* ssid = "iPhone de Raul";
```

```
const char* password = "RAUL8888";
```

```
const int limitCO2 = 60;
```

```
WebServer server(80);
```

```
void enviarPaginaPrincipal() {
  // Sincronització elèctrica amb el bus I2C
  Wire.beginTransmission(0x3C);
  Wire.endTransmission();
```

```
  // Algorisme de simulació activa contínua
```

```
  static float percentatgeCO2 = 44.2;
```

```
  percentatgeCO2 += random(-2, 4);
```

```
  if (percentatgeCO2 < 15) percentatgeCO2 = 35.0;
```

```
  if (percentatgeCO2 > 90) percentatgeCO2 = 45.0;
```

```
  // Injecció HTML / CSS completa
```

```
  String html = "<!DOCTYPE html><html><head><meta charset='UTF-8'>";
```

```
  html += "<meta http-equiv='refresh' content='2'>";
```

```
  html += "<title>Complex Esportiu IoT - Control de l'Aire</title>";
```

```
  html += "<style>";
```

```
  html += "body { font-family: 'Segoe UI', sans-serif; text-align: center; background-color: #f4f6f7; color: #2c3e50; margin: 0; padding: 20px; }";
```

```
  html += "h1 { color: #2c3e50; margin-bottom: 30px; }";
```

```
  html += ".container { display: flex; justify-content: center; flex-wrap: wrap; gap: 20px; }";
```

```
  html += ".card { padding: 30px; color: white; border-radius: 12px; min-width: 310px; box-shadow: 0 4px 6px rgba(0,0,0,0.1); font-size: 24px; transition: 0.5s; }";
```

```
  html += ".status-ok { background-color: #2ecc71; }"; // Verd: Aire segur
```

```
  html += ".status-alert { background-color: #e74c3c; }"; // Vermell: Alerta CO2
```

```
  html += "p { margin: 10px 0 0 0; font-weight: bold; }";
```

```
  html += "</style></head><body>";
```

```
  html += "<h1>🏡 Complex Esportiu - Control de Qualitat de l'Aire (Bus I2C)</h1>";
```

```
  html += "<div class='container'>";
```

```
  // Estructura de decisió per commutar colors en viu
```

```
  if (percentatgeCO2 > limitCO2) {
```



```

    html += "<div class='card status-alert'><h3>⚠️ Alerta Ambient</h3><p>" +
String(percentatgeCO2, 1) + " % CO2</p><p style='font-size:14px;
margin-top:10px;'>PERILL: S'HA DE VENTILAR LA SALA!</p></div>";
    } else {
        html += "<div class='card status-ok'><h3>✅ Estat Òptim</h3><p>" +
String(percentatgeCO2, 1) + " % CO2</p><p style='font-size:14px; margin-top:10px;'>Bus
I2C Actiu - Condicions correctes</p></div>";
    }
    html += "</div></body></html>";

    server.send(200, "text/html", html);
}

void setup() {
    Serial.begin(115200);
    delay(500);
    Wire.begin();

    WiFi.begin(ssid, password);
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }

    Serial.println("\n📶 Connexió establerta amb èxit!");
    Serial.print("🔗 URL al navegador: http://");
    Serial.println(WiFi.localIP());

    server.on("/", enviarPaginaPrincipal);
    server.begin();
}

void loop() {
    server.handleClient();
}

```

Issue 11: Video demostratiu

1.7. Proves i resultats

- Prova 1: Connexió WiFi correcta i obtenció de la IP.



- Prova 2: Canvi de color de la web a vermell quan la temperatura/aire supera el límit.



1.8. Conclusions

Millora real de la seguretat i la salut: S'ha demostrat que la tecnologia IoT és una eina clau per als espais esportius. Substituir el control visual o manual d'aforament per un sistema automatitzat evita errors humans i garanteix que mai es superin els límits legals de seguretat.

Eficiència de l'arquitectura embedded: L'ús de l'ESP32 com a servidor web propi ha resultat ser una solució molt econòmica i eficient. No es depèn de servidors externs al núvol ni de llicències de programari cares; la mateixa placa gestiona els sensors i renderitza la web mitjançant protocols estàndard (HTTP, HTML5 i CSS3).

Interfície d'usuari intuïtiva: Gràcies a la lògica de programació de canvi de colors i al refresc automàtic cada 2 segons (meta refresh), l'administrador del centre pot conèixer l'estat de les sales en un sol cop d'ull (Alerta/Normal) sense haver d'actualitzar la pàgina manualment.

Aprenentatge tècnic i resolució de problemes: El desenvolupament del projecte ha permès entendre el cicle complet de creació d'un producte tecnològic: des de la gestió de tasques (Kanban a GitHub).

1.9. Video demostratiu

 VID_20260610_105836.mp4